Metro ridership forecasting with periodic and calendar context

Extending PVCGN (Liu et al., IEEE T-ITS 2022): the paper, the research gap we chose, and a guide to every file in the project

Nitin Bharti

Route map for this talk



1 — The original paper

What PVCGN does and why

2 — Research gap

What PVCGN misses, with evidence

3 — Our solution

PT-PVCGN and how we test it fairly

4 — Code walkthrough

Every folder and file, and its purpose

5 — Results

Final test results and conclusion

1

The original paper: PVCGN

Physical-Virtual Collaboration Graph Network — Liu, Chen, Wu, Zhen, Li, Lin · IEEE T-ITS 23(4), 2022

The problem: forecast every station's next hour

8.8 M

trips per day on the Shanghai metro (2016 data)

15 min

time resolution: inflow and outflow per station

1 hour

forecast horizon: the next four 15-minute slots

Why it matters: operators use these forecasts to add trains before crowds build, place staff at busy stations, control platform crowding and warn passengers. A wrong forecast at rush hour can mean dangerous overcrowding or wasted trains.

Inflow = passengers entering a station. **Outflow** = passengers leaving it. Both are counted every 15 minutes from smart-card taps.

PVCGN's idea: describe the metro with three graphs

Physical graph

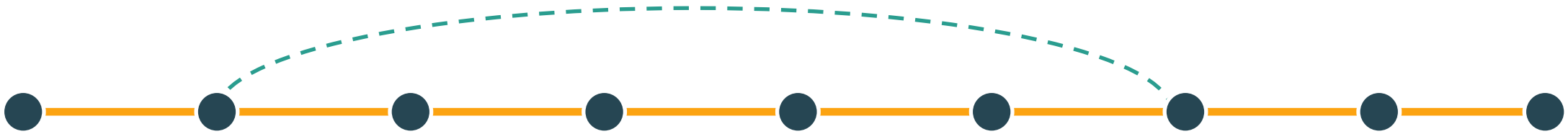
Stations joined if they are next to each other on a metro line. Crowds physically move along the tracks.

Similarity graph (virtual)

Stations joined if their daily ridership curves look alike (dynamic time warping), even when far apart — e.g. two office districts.

Correlation graph (virtual)

Stations joined if many passengers travel between them (origin–destination ratios). If most of A's riders go to B, A helps predict B.



A metro line (solid) and a virtual similarity edge (dashed) linking two distant stations

How PVCGN forecasts



Last hour

4 × 15-min inflow / outflow for all stations

Encoder

2 stacked CGRM layers read the hour step by step

Decoder

2 CGRM layers produce one 15-min step at a time

Next hour

4 forecasts: 15, 30, 45, 60 min ahead

Inside a CGRM: GC-GRU (local)

A gated recurrent unit whose gates use graph convolution: each station mixes its own state with its neighbours' in all three graphs, every time step.

Inside a CGRM: FC-GRU (global)

A fully-connected GRU over all stations at once captures the city-wide trend; its output is fused with the local GC-GRU state.

Two benchmark datasets released with the paper

SHMetro — Shanghai

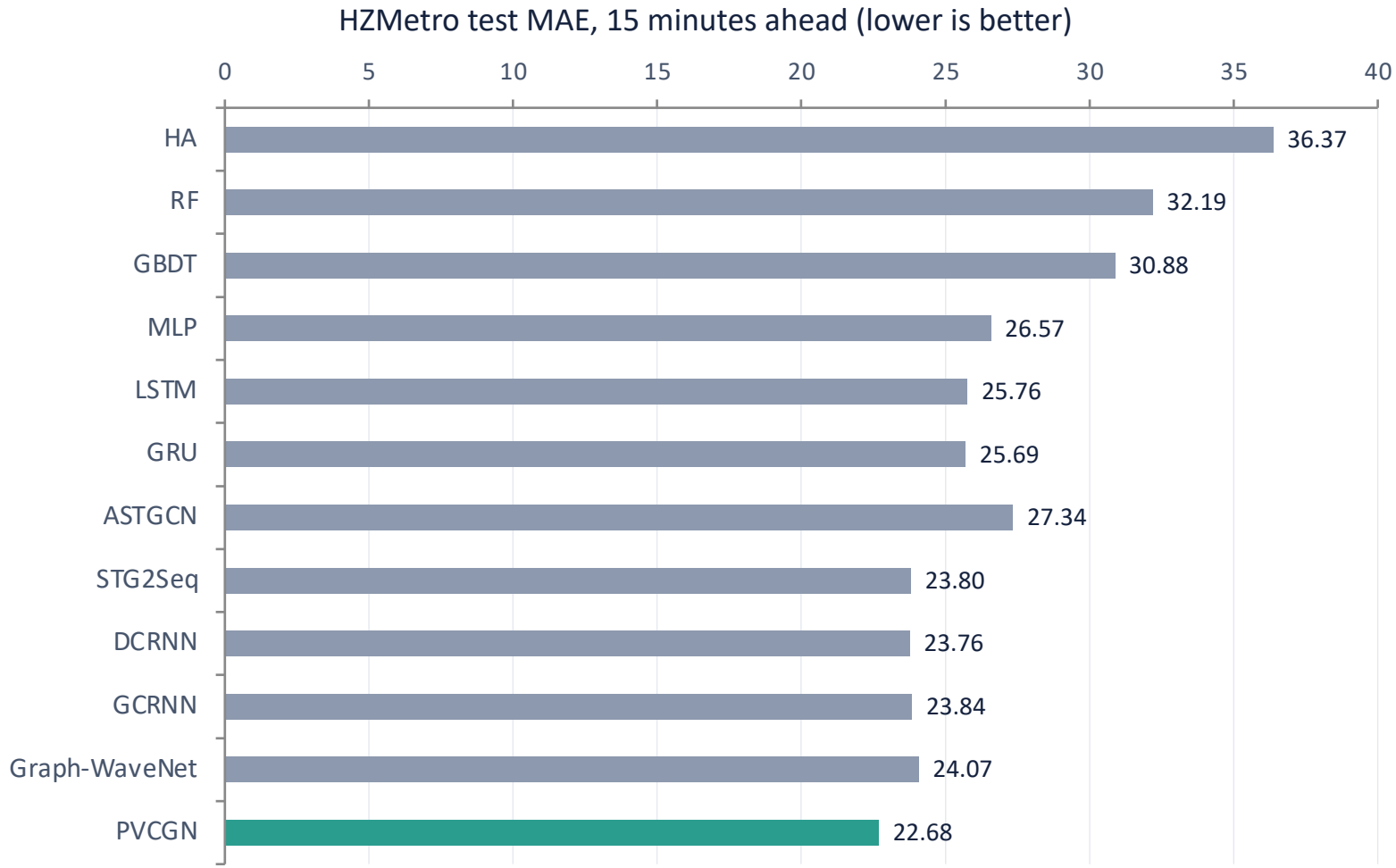
- 288 stations, 958 physical edges
- 811.8 M transactions, Jul–Sep 2016
- Train Jul 1–Aug 31 · Val Sep 1–9 · Test Sep 10–30
- Test period includes a holiday (Sep 15–17)

HZMetro — Hangzhou

- 80 stations, 248 physical edges
- 2.35 M trips per day, January 2019
- Train Jan 1–18 · Val Jan 19–20 · Test Jan 21–25
- Validation days are a weekend; test days are weekdays

Service hours 05:30–23:30 → 73 slots of 15 minutes per day, 66 forecast windows per day. Splits are by date, so the model is always tested on days after the ones it trained on.

Paper result: PVCGN beat all compared methods



What the paper reports

- Best MAE, RMSE and MAPE on both cities at all 4 horizons
- Also best at rush hours and at the busiest stations
- Numbers from Table III of the paper

2

The research gap

PVCGN never looks at the clock, the calendar or earlier days

Gaps we found — and the one we chose

Gap 1 · CHOSEN

No periodic or calendar context: PVCGN sees only the last hour, never the time of day, weekday/holiday, or earlier days.

Gap 2 · future work

Similarity and correlation graphs are fixed all day, though which stations behave alike changes between peaks.

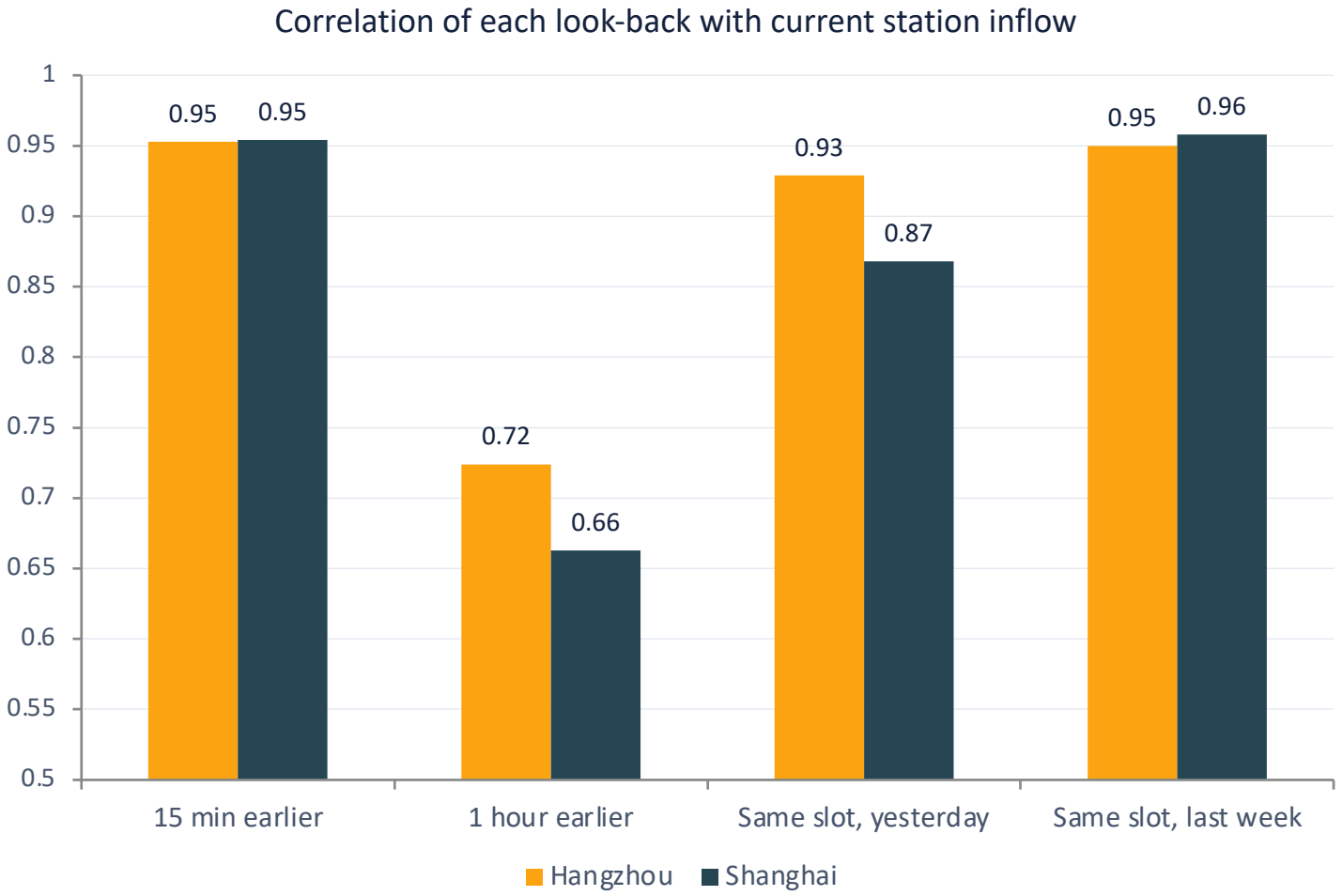
Gap 3 · future work

Point forecasts only: no prediction intervals, although busy stations are far noisier.

Gap 4 · handled in our tests

Missing baselines (same-time-last-week, boosting with periodic features), single runs, holidays not analysed.

Evidence: past days predict far better than one hour ago



Why this matters

To forecast 60 minutes ahead, PVCGN's newest input is already an hour old ($r \approx 0.66–0.72$). The same slot last week is much closer ($r \approx 0.95$).

And on training data

Same slot on the last comparable day vs. ridership 60 min ahead: $r = 0.96$ (HZ), 0.99 (SH). The last observation: $r = 0.70$ / 0.66.

3

Our solution: PT-PVCGN

PVCGN + Periodic inputs + Time (calendar) context

PT-PVCGN adds two switchable parts to PVCGN

P — periodic reference inputs

- Same time slots on the most recent day of the same type (working / non-working)
- …and on the same weekday one week earlier
- Only past days: no future information

T — calendar context

- Learned embeddings of time of day, day of week and working day
- Official calendar: holidays and make-up working days (e.g. Sun 18 Sep 2016)



Ablation study: train all three under identical settings to measure what each part adds. With P and T switched off the code is exactly PVCGN.

Which models we use, and why

PVCGN (reproduced)

The published state of the art on these datasets — the model we must beat. Re-built exactly so the comparison is fair.

PT-PVCGN (ours)

PVCGN + periodic inputs + calendar. Tests the research gap directly while keeping all of PVCGN's strengths.

+P only (ablation)

PVCGN + periodic inputs, without calendar. Separates what each addition contributes.

Simple baselines

Persistence, seasonal naive, historical average. Show how much of the task is plain repetition.

Ridge · gradient boosting

Cheap models given the same periodic features. Show whether a deep graph model is worth its cost.

Why graph + GRU at all?

A metro is a graph (space) and ridership is a sequence (time): graph convolution handles space, a GRU handles time.

How PT-PVCGN makes a forecast



Inputs

Each 15-min step, per station: in/out now, same slot on the reference day and last week, time-of-day and day-type codes

Graph convolution

Each station mixes its inputs with neighbours in the physical, similarity and correlation graphs

GC-GRU memory

Keeps 256 numbers per station, updated step by step over the last hour

FC-GRU (global)

Summarises the whole network and fuses the city-wide trend back in

Decoder

4 steps; each sees its target slot's reference values and time, then outputs in/out

The key idea

The model already knows what normally happens at, say, 08:45 on a working day, so it only has to learn how today differs. Steps 2–4 are unchanged PVCGN; our additions enter at steps 1 and 5.

How the models are trained

Loss

Mean absolute error on real passenger counts — the paper's main metric.

Optimiser

Adam, learning rate 0.001, cut ×10 at epochs 100 and 150 (paper schedule); gradient clipping at 5.

Curriculum learning

Early on, the decoder gets the true previous values; gradually it must use its own predictions.

Picking the model

Validation MAE after every epoch; keep the best; stop after 50 epochs without improvement.

Repeats

3 seeds per model on Hangzhou, 2 on Shanghai; results reported as mean ± std.

Budget

Hangzhou: 200 epochs for every model (~1–1.5 h per run). Shanghai: 30 epochs (~5–8 h per run).

How we make the comparison fair



Problem defined

passed

Data and plan ready

passed

Validation results

passed

Final test + report

passed

Same data, same metric

Paper's exact date splits and its MAE / RMSE / MAPE code. PVCGN re-implemented and checked equal to the authors' code.

Strong baselines

Persistence, seasonal naive, historical average, ridge and gradient boosting with periodic features.

Test set used once

All choices made on validation. 2–3 seeds per model. Every result reported honestly, good or bad.

4

Code walkthrough

Every folder and file in Desktop/Minor, and why it exists

Tech stack

Python 3.14

The language for all data, model and analysis code.

PyTorch 2.14 + Apple MPS

Builds and trains the neural networks on the Mac's built-in GPU.

NumPy 2.5 · pandas 3.1

Arrays and tables: loading data, building features, computing metrics.

scikit-learn 1.9

Ridge regression and gradient-boosting baselines.

matplotlib 3.11

Every analysis and results figure.

marimo 0.25

Notebooks (01_eda, 02_prep, 03_model) that also run as plain scripts.

torch_geometric 2.8

Only to run the authors' original code in the equivalence test.

git

Version history, with a checkpoint at every review gate.

LaTeX · Node + pptxgenjs

The paper (IEEE template) and this presentation.

Project folder map: Desktop/Minor

`metro/`

Our Python package: data loading, features, model, training, metrics, baselines.

`ml_pipeline/`

The step-by-step workflow: notebooks, guard, progress log, figures, saved splits and run results.

`reference_PVCGN/`

The authors' original code and data, cloned from GitHub. Not modified.

`tests/`

Automatic check that our PVCGN equals the authors' version.

`paper/`

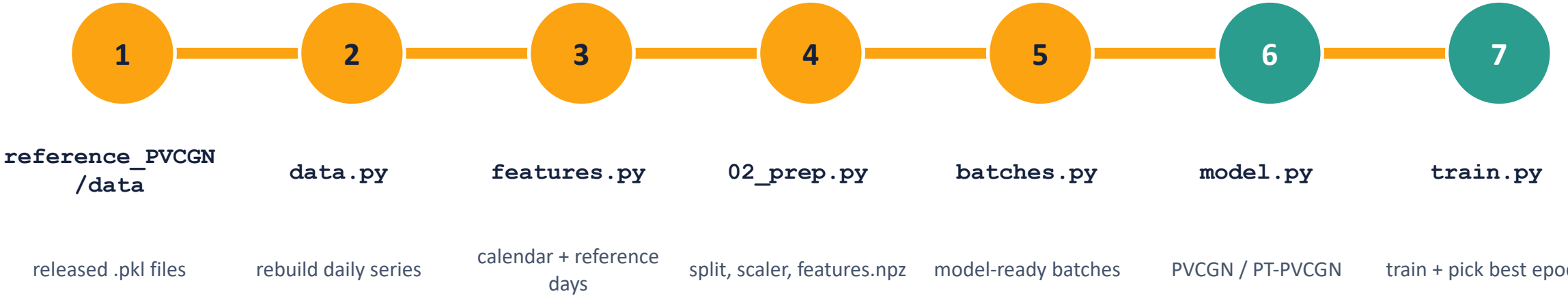
Draft of the new research paper (LaTeX) and its references.

`presentation/`

This deck and the script that builds it.

Also at the top level: run_queue.sh (runs all training jobs), .gitignore, .venv/ (Python environment), checkpoints/ (saved model weights).

How the files connect



Side branch: baselines

baselines.py fits simple models on the same features; 03_model.py scores them.

Shared: metrics.py

One MAE / RMSE / MAPE implementation used by every model, identical to the paper's.

Outputs

ml_pipeline/runs/*.json (scores per epoch) and checkpoints/*.pt (best weights).

metro/ — data side

`data.py`

- Loads the authors' train / val / test .pkl files and graphs
- Rebuilds continuous days (73 slots × stations × in/out) from overlapping windows, and checks the rebuild exactly
- long_table(): one row per station per 15 min, used for analysis

`features.py`

- Official calendar: holidays and make-up working days
- Working-day flag for every date
- Reference days: last earlier day of the same type, and same weekday a week earlier
- Only uses past days

`batches.py`

- Loads the prepared features.npz and scaler
- Scales counts (z-score from training data)
- Builds each batch: last hour, targets, reference-day slots, time slot, weekday, working day

metro/ — model side

`model.py`

- PVCGN rewritten in plain PyTorch: graph convolution, GC-GRU + FC-GRU, encoder–decoder
- Switches periodic= and temporal= add our P and T parts

`train.py`

- Trains one model (city, variant, seed) on training days
- Scores validation every epoch; keeps the best epoch
- Never reads the test set

`metrics.py`

- MAE, RMSE and MAPE, ignoring zero-passenger slots — same rule as the paper's code
- Per horizon (15/30/45/60 min)

`baselines.py`

- Persistence, seasonal naive, historical average
- Ridge and gradient boosting on recent + reference-day + calendar features

ml_pipeline/guard.py — the safety rails

A helper library that stops the most common mistakes in machine-learning projects, above all data leakage: letting test information influence training or model choice.

`profile()`

Inspects the raw data: size, column types, missing values, duplicates, leakage suspects. Writes data_profile.json.

`eda_figures()` · `fig()`

Saves every chart to figures/ and logs a one-line explanation in PIPELINE.md.

`split()`

Splits by time (refuses random splits on dated data), checks no row is in two splits, and freezes a fingerprint of the test set.

`final_test()`

Scores the final model on the test set exactly once; refuses a second try or a tampered test set; logs the score.

ml_pipeline/ — notebooks and progress log

`01_eda.py`

- Steps 1–2: inspects the data and draws the analysis charts
- Found the periodicity, weekday/weekend and holiday patterns

`02_prep.py`

- Steps 4–8: cleaning checks, calendar and reference days, guard split, scaler
- Saves features.npz and scaler.json per city

`03_model.py`

- Step 9 onward: scores all baselines on validation
- Will compare baselines, PVCGN and PT-PVCGN

`PIPELINE.md`

The project's logbook: status of all 16 steps, review sign-offs, research gaps, decisions, figure notes

ml_pipeline/ — saved results and records

`data_profile.json`

Output of guard.profile(): data size, column kinds, leakage checks

`figures/`

Every chart, numbered by pipeline step (e.g. 02_periodicity_lags.png)

`hz/ and sh/`

Per city: features.npz (prepared data), scaler.json, split_state.json (frozen test fingerprint)

`runs/`

- One .json per training run: settings and scores per epoch
- baselines_val.json and logs/

Other project files

`tests/test_equivalence.py`

- Copies the same weights into our PVCGN and the authors' PVCGN
- Checks both give the same output (to 1e-5)

`run_queue.sh`

- Runs every training job one after another
- Skips jobs already finished, so it can restart safely

`paper/main.tex` · `refs.bib`

- Draft of the new paper (IEEE format) and its bibliography
- Results sections fill in after the final test

`.gitignore` · `.venv/` · `checkpoints/`

Files kept out of git; the Python environment; saved model weights

reference_PVCGN/ — the authors' original code

ggnn/multigraph.py

PVCGN model: GGRU cell (GC-GRU + FC-GRU) and the encoder–decoder

ggnn/rgcn.py

Relational graph convolution layer (one weight per graph)

ggnn_train.py

Original training script; YAML configs in data/model/

ggnn_evaluation*.py

Original evaluation: all data, rush hours, top stations

ggnn_utils.py · lib/

Batching (torch_geometric), data loading, scaler, metrics

data/

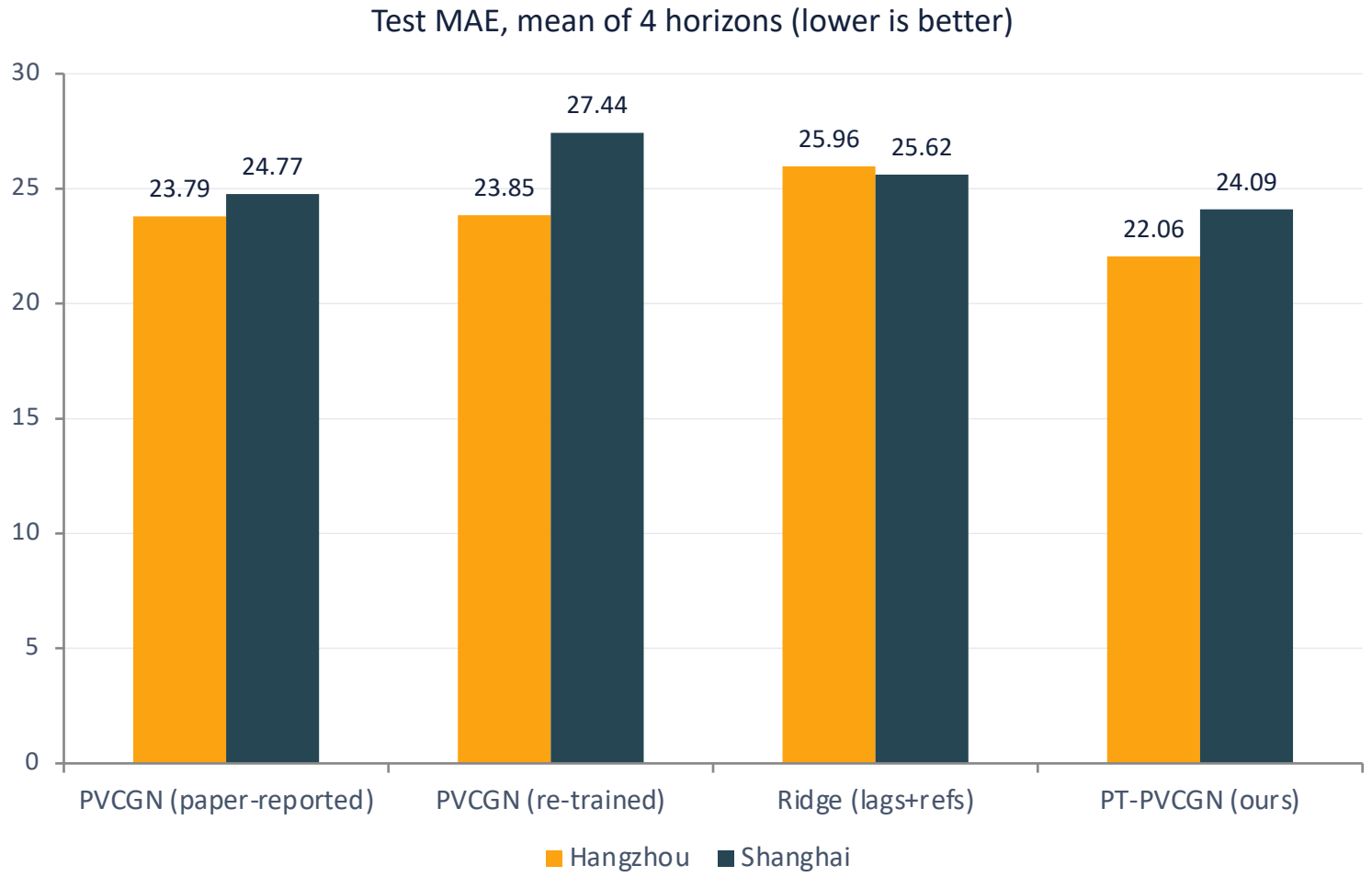
Released datasets (.pkl) and the three graphs per city

5

Final results

Test set of each city evaluated once, all models together, after every choice was made on validation

Test MAE: PT-PVCGN beats PVCGN on both cities



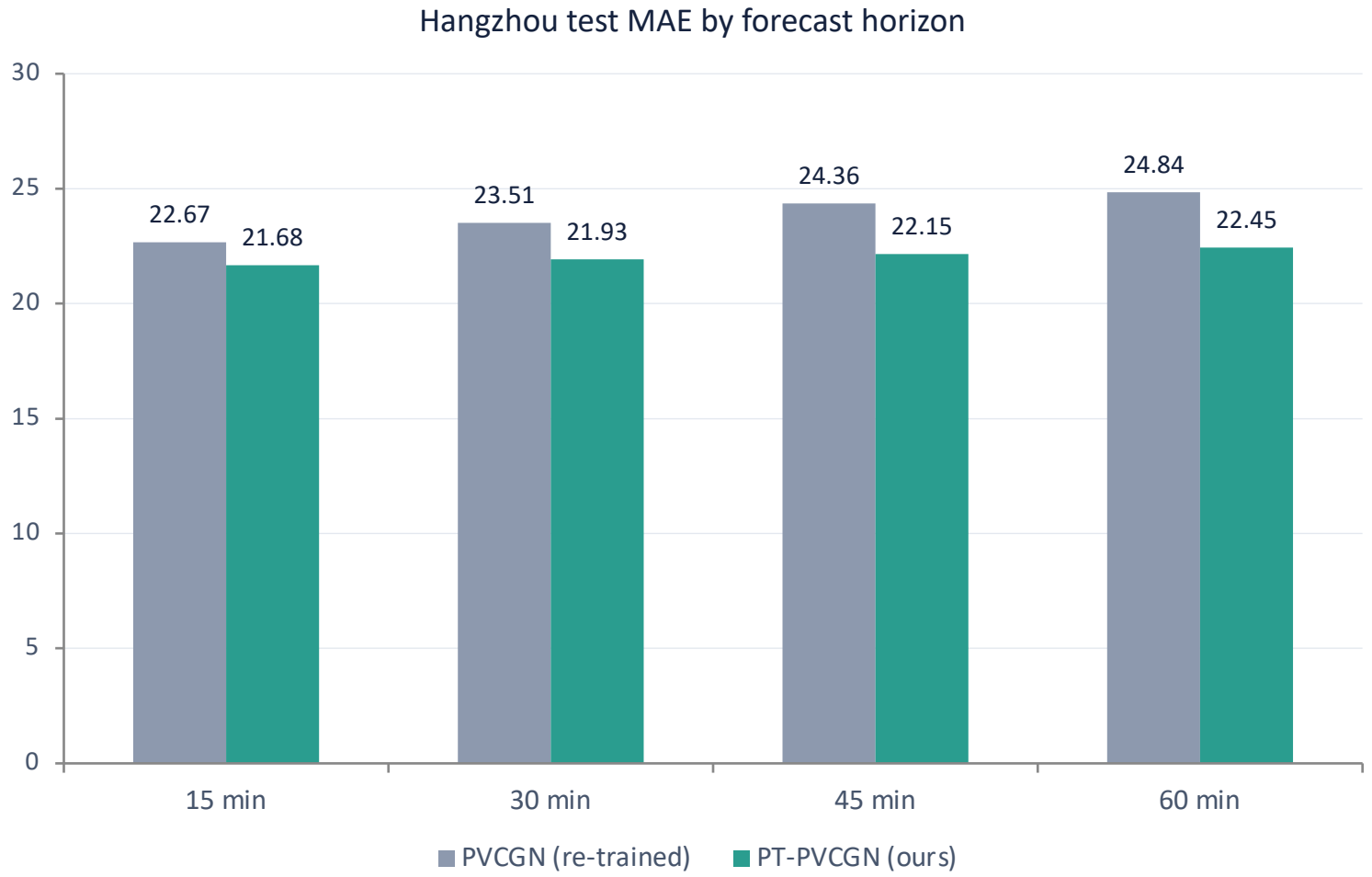
Hangzhou: 7.5% lower error

Same 200-epoch training for both. Our PVCGN matches the paper (23.85 vs 23.79).

Shanghai: 12.2% lower error

Both trained 30 epochs. Ours also beats the paper's 200-epoch PVCGN (24.09 vs 24.77).

The gain grows the further ahead we forecast



Exactly what the gap predicts

- 15 min ahead: 4.4% lower error
- 60 min ahead: 9.6% lower error
- The further ahead, the less the last hour helps — and the more the same time on a past day helps

Shanghai: whole test set

Time	Metric	HA	SN	Ridge	GBDT	DCRNN†	GCRNN†	Graph-WaveNet†	PVCGN†	PVCGN	PT-PVCGN
15 min	RMSE	67.38	76.89	47.52	53.97	46.02	46.09	46.98	**44.97**	47.67	46.08
	MAE	32.25	35.73	24.56	25.72	24.04	24.26	24.91	23.29	25.19	**22.98**
	MAPE	20.12%	23.28%	17.29%	**16.80%**	17.82%	18.06%	20.05%	16.83%	19.27%	17.19%
30 min	RMSE	67.36	76.99	49.45	55.51	49.90	50.12	51.64	**47.83**	52.31	48.68
	MAE	32.12	35.75	25.15	26.21	25.23	25.42	26.53	24.16	26.75	**23.78**
	MAPE	20.25%	23.59%	18.24%	**17.21%**	18.35%	18.73%	20.38%	17.23%	19.94%	17.69%
45 min	RMSE	67.31	77.15	52.03	57.71	54.92	54.87	58.50	52.02	56.55	**51.09**
	MAE	31.91	35.76	26.00	26.86	26.76	26.92	28.78	25.33	28.21	**24.51**
	MAPE	20.37%	23.90%	19.68%	**17.65%**	19.30%	19.81%	21.99%	17.92%	20.56%	18.39%
60 min	RMSE	67.15	77.35	54.30	59.37	58.83	58.67	65.08	55.27	60.86	**52.84**
	MAE	31.70	35.78	26.78	27.43	28.01	28.18	30.90	26.29	29.63	**25.08**
	MAPE	20.43%	24.11%	20.75%	**18.09%**	20.44%	21.07%	24.36%	18.69%	21.97%	18.92%

† reported in the original PVCGN paper (Liu et al., 2022), same test split, not re-run. HA: historical average · SN: seasonal naive · Bold: best in row · Teal columns: our trained models (mean over seeds).

Hangzhou: whole test set

Time	Metric	HA	SN	Ridge	GBDT	DCRNN†	GCRNN†	Graph-WaveNet†	PVCGN†	PVCGN	+P	PT-PVCGN
15 min	RMSE	53.89	52.17	40.88	43.83	40.39	40.24	40.78	37.76	37.69	**34.99**	35.37
	MAE	30.01	27.98	24.44	25.87	23.76	23.84	24.07	22.68	22.67	**21.41**	21.68
	MAPE	15.81%	16.43%	14.50%	14.40%	14.00%	14.08%	14.27%	13.70%	13.58%	**12.90%**	13.12%
30 min	RMSE	53.86	52.21	42.74	45.19	42.57	41.95	42.80	39.34	39.40	**35.15**	35.73
	MAE	30.03	28.03	25.41	26.73	25.22	25.14	25.48	23.33	23.51	**21.62**	21.93
	MAPE	15.93%	16.71%	15.44%	14.89%	14.99%	14.86%	15.23%	13.81%	13.88%	**13.22%**	13.30%
45 min	RMSE	53.75	52.17	44.86	46.12	46.26	45.53	45.84	40.95	40.94	**35.44**	36.12
	MAE	29.96	27.98	26.38	27.24	26.97	26.82	27.15	24.22	24.36	**21.84**	22.15
	MAPE	16.12%	17.09%	16.79%	15.39%	16.19%	16.05%	17.34%	14.45%	14.57%	13.78%	**13.73%**
60 min	RMSE	53.65	52.16	47.44	47.66	49.35	50.28	49.89	42.61	42.18	**36.02**	36.79
	MAE	29.83	27.90	27.60	27.87	28.47	28.75	29.14	24.93	24.84	**22.03**	22.45
	MAPE	16.55%	17.70%	19.32%	16.18%	18.16%	17.89%	19.37%	15.49%	15.55%	**14.29%**	14.52%

† reported in the original PVCGN paper (Liu et al., 2022), same test split, not re-run. HA: historical average · SN: seasonal naive · Bold: best in row · Teal columns: our trained models (mean over seeds).

Shanghai: rush hours

Time	Metric	HA	SN	Ridge	GBDT	DCRNN†	GCRNN†	Graph-WaveNet†	PVCGN†	PVCGN	PT-PVCGN
15 min	RMSE	82.41	97.93	**62.03**	74.71	67.50	66.21	68.41	65.04	62.89	63.38
	MAE	41.10	47.74	32.68	35.32	37.92	37.94	39.17	36.46	34.27	**32.24**
	MAPE	16.83%	19.49%	13.94%	14.01%	13.93%	14.07%	14.14%	**13.16%**	14.58%	13.85%
30 min	RMSE	82.41	97.93	**63.93**	76.01	73.07	73.63	78.98	68.85	68.03	66.49
	MAE	41.10	47.74	33.38	35.92	40.16	40.26	43.54	37.77	36.88	**33.34**
	MAPE	16.83%	19.49%	14.27%	14.29%	14.33%	14.44%	14.79%	**13.41%**	15.29%	14.29%
45 min	RMSE	82.41	97.93	**67.33**	78.39	79.42	79.88	87.66	73.85	75.50	69.80
	MAE	41.10	47.74	34.65	37.13	41.92	41.65	46.26	38.84	40.32	**34.59**
	MAPE	16.83%	19.49%	14.79%	14.72%	15.29%	15.29%	15.95%	**14.06%**	16.30%	14.84%
60 min	RMSE	82.41	97.93	**70.17**	79.49	79.98	81.37	90.19	74.41	83.74	71.90
	MAE	41.10	47.74	35.71	37.85	41.23	41.27	46.35	38.12	43.98	**35.41**
	MAPE	16.83%	19.49%	15.17%	**15.06%**	16.58%	16.62%	17.66%	15.08%	17.63%	15.24%

† reported in the original PVCGN paper (Liu et al., 2022), same test split, not re-run. HA: historical average · SN: seasonal naive · Bold: best in row · Teal columns: our trained models (mean over seeds).

Hangzhou: rush hours

Time	Metric	HA	SN	Ridge	GBDT	DCRNN†	GCRNN†	Graph-WaveNet†	PVCGN†	PVCGN	+P	PT-PVCGN
15 min	RMSE	57.25	69.22	47.19	51.11	54.17	55.51	56.98	49.79	47.38	**45.13**	45.78
	MAE	33.11	35.79	29.54	30.82	35.08	35.68	37.19	32.63	29.62	**28.13**	28.56
	MAPE	10.64%	11.89%	10.17%	10.15%	10.37%	10.36%	10.84%	9.72%	10.12%	**9.65%**	9.75%
30 min	RMSE	57.25	69.22	47.74	51.36	58.27	57.34	59.71	51.63	49.00	**44.84**	45.78
	MAE	33.11	35.79	29.99	31.22	37.48	37.31	38.94	33.30	30.69	**28.11**	28.70
	MAPE	10.64%	11.89%	10.34%	10.26%	10.69%	10.54%	11.04%	**9.52%**	10.32%	9.66%	9.79%
45 min	RMSE	57.25	69.22	49.42	52.22	61.83	59.54	59.83	51.45	51.09	**44.95**	46.18
	MAE	33.11	35.79	30.97	31.89	37.95	37.58	38.75	32.73	32.14	**28.25**	29.04
	MAPE	10.64%	11.89%	10.68%	10.51%	11.16%	11.16%	11.83%	9.88%	10.76%	**9.74%**	9.95%
60 min	RMSE	57.25	69.22	50.84	53.18	59.52	58.88	59.96	51.09	53.59	**45.59**	46.88
	MAE	33.11	35.79	31.76	32.26	36.27	35.94	37.49	31.43	33.10	**28.44**	29.41
	MAPE	10.64%	11.89%	10.91%	10.58%	11.94%	11.93%	12.35%	10.43%	10.96%	**9.80%**	10.02%

† reported in the original PVCGN paper (Liu et al., 2022), same test split, not re-run. HA: historical average · SN: seasonal naive · Bold: best in row · Teal columns: our trained models (mean over seeds).

Shanghai: busiest 1/4 of stations

Time	Metric	HA	SN	Ridge	GBDT	DCRNN†	GCRNN†	Graph-WaveNet†	PVCGN†	PVCGN	PT-PVCGN
15 min	RMSE	118.06	137.31	81.01	94.06	84.04	86.09	76.93	**74.80**	77.70	78.54
	MAE	63.21	71.47	45.73	48.42	44.98	45.89	43.31	**41.38**	43.44	41.86
	MAPE	16.06%	17.99%	12.15%	11.93%	13.76%	14.12%	11.68%	**10.62%**	11.67%	11.08%
30 min	RMSE	117.96	137.28	84.48	96.70	88.52	89.89	84.23	**79.43**	85.36	83.20
	MAE	62.92	71.35	47.09	49.56	46.80	47.50	46.32	**43.05**	46.49	43.51
	MAPE	16.44%	18.48%	13.80%	12.63%	14.47%	14.82%	13.12%	**11.46%**	13.11%	11.93%
45 min	RMSE	117.90	137.43	89.35	100.91	97.75	96.81	96.35	**87.32**	93.11	87.81
	MAE	62.57	71.31	49.06	51.29	49.84	50.08	50.76	45.67	49.74	**45.28**
	MAPE	16.67%	18.93%	16.51%	13.35%	16.32%	16.70%	15.50%	**12.48%**	14.54%	13.00%
60 min	RMSE	117.79	137.78	93.67	104.17	106.03	102.93	109.26	93.59	101.48	**91.05**
	MAE	62.29	71.42	50.95	52.77	53.30	52.79	55.74	48.02	53.21	**46.58**
	MAPE	16.78%	19.20%	18.87%	13.91%	17.89%	18.16%	18.27%	**13.61%**	16.30%	13.80%

† reported in the original PVCGN paper (Liu et al., 2022), same test split, not re-run. HA: historical average · SN: seasonal naive · Bold: best in row · Teal columns: our trained models (mean over seeds).

Hangzhou: busiest 1/4 of stations

Time	Metric	HA	SN	Ridge	GBDT	DCRNN†	GCRNN†	Graph-WaveNet†	PVCGN†	PVCGN	+P	PT-PVCGN
15 min	RMSE	93.22	89.55	68.22	72.26	66.18	65.29	65.87	60.56	60.56	**56.06**	56.97
	MAE	57.64	49.22	44.25	46.14	41.22	40.93	40.72	38.29	38.34	**36.05**	36.92
	MAPE	13.49%	12.74%	10.90%	11.45%	10.72%	10.59%	10.39%	9.97%	9.94%	**9.43%**	9.63%
30 min	RMSE	93.09	89.59	71.77	74.45	69.36	67.29	68.92	63.77	63.58	**56.11**	57.31
	MAE	57.61	49.32	46.60	47.82	43.92	43.07	43.21	39.93	40.03	**36.38**	37.28
	MAPE	13.66%	13.00%	11.52%	12.03%	11.51%	11.42%	11.40%	10.34%	10.38%	**9.71%**	9.89%
45 min	RMSE	92.80	89.45	75.58	75.98	75.16	72.42	72.96	65.99	65.82	**56.35**	57.87
	MAE	57.34	49.14	48.73	48.80	46.99	45.93	45.71	41.75	41.65	**36.63**	37.68
	MAPE	13.99%	13.45%	12.55%	12.53%	12.74%	12.46%	13.32%	11.28%	11.19%	**10.22%**	10.36%
60 min	RMSE	92.59	89.39	80.39	78.81	79.79	80.34	79.70	69.25	67.97	**57.48**	59.21
	MAE	57.04	48.94	51.51	49.99	49.50	49.40	49.77	43.16	42.56	**37.04**	38.49
	MAPE	14.66%	14.28%	15.10%	13.65%	15.00%	14.74%	16.56%	12.54%	12.49%	**10.96%**	11.51%

† reported in the original PVCGN paper (Liu et al., 2022), same test split, not re-run. HA: historical average · SN: seasonal naive · Bold: best in row · Teal columns: our trained models (mean over seeds).

What the ablation and error analysis show

Periodic inputs carry the gain

+P alone: test MAE 21.72 on Hangzhou — the lowest error under equal (200-epoch) training.

Calendar part adds nothing

+P+T (22.06) is not better than +P. Reported honestly in the paper.

Wins where it matters

Rush hours and busiest stations, both cities; Shanghai working days 24.48 vs 29.61.

Weak on irregular days

Holiday 24.49 vs PVCGN 21.48; make-up Sunday 33.69 vs 29.41.

Conclusion

Giving PVCGN the same time slots on comparable past days cuts forecast error by 7.5% (Hangzhou) and 12.2% (Shanghai). Next: holiday-aware reference days, longer Shanghai training, and submission of the paper.